05 정렬

- **선택 정렬** Selection Sort
  - 매 패스마다 최댓값(혹은 최솟값)을 찾아서 마지막 원소와 교환

시작	8	31	48	73	3	65	20	29	11	15
1	8	31	48	15	3	65	20	29	11	73

남은 원소에서 최댓값인 73을 찾고 마지막 원소와 교환

1	8	31	48	15	3	65	20	29	11	73
2	8	31	48	15	3	11	20	29	65	73

남은 원소에서 최댓값인 65을 찾고 마지막 원소와 교환

	0	1	2	3	4	5	6	7	8	9
시작	8	31	48	73	3	65	20	29	11	15
1	8	31	48	15	3	65	20	29	11	73
2	8	31	48	15	3	11	20	29	65	73
3	8	31	29	15	3	11	20	48	65	73
4	8	20	29	15	3	11	31	48	65	73
5	8	20	11	15	3	29	31	48	65	73
6	8	3	11	15	20	29	31	48	65	73
7	8	3	11	15	20	29	31	48	65	73
8	8	3	11	15	20	29	31	48	65	73
9	3	8	11	15	20	29	31	48	65	73

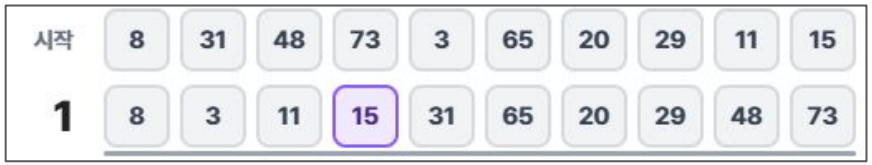
아직 정렬되지 않은 원소 / 이번 패스에서 자리를 맞바꾼 원소

자리가 확정된 원소 / 이번 패스에서 최솟값을 찾은 구간 (표 아래 밑줄)

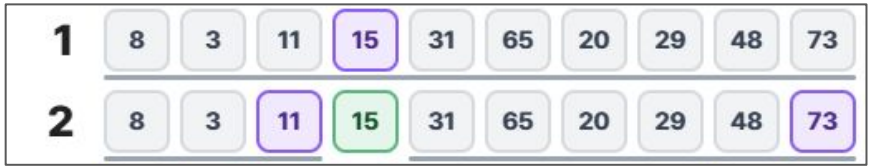
Copyright ⓒ 2026 by Lee San. All rights reserved.

05 정렬

- **퀵 정렬** Quick Sort
  - 기준 원소 하나를 정해 그보다 작은 값은 왼쪽, 큰 값은 오른쪽으로 몰아 기준 원소의 자리를 확정한 뒤, 양쪽 구간에 같은 일을 재귀적으로 반복하는 정렬

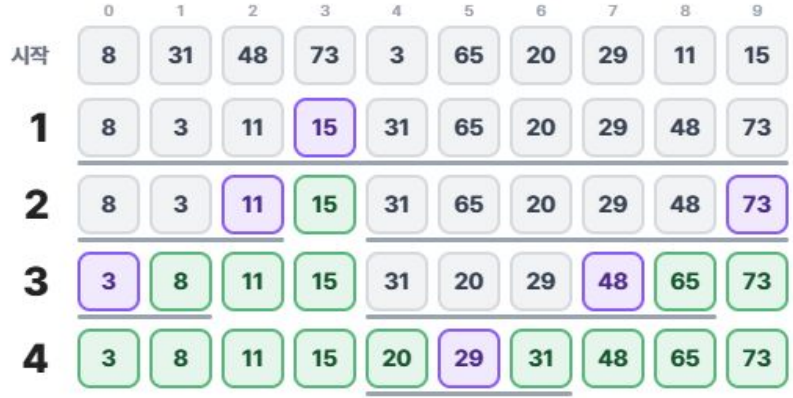


배열 전체를 기준값 15로 분할해 15보다 작은 8, 3, 11을
앞으로 몰아넣고 15를 A[3]에 확정



15로 갈라진 두 구간 A[0..2]와 A[4..9]를 동시에 분할하여
기준값 11과 73을 확정





Copyright ⓒ 2026 by Lee San. All rights reserved.

프로그래밍I
탐색 기반 알고리즘

02 선형 구조의 탐색



01 선형 구조

- **선형 구조**
  - 자료의 순서를 유일하게 결정할 수 있는 형태의 구조
  - i 번째 자료 를 탐색한 다음, i+1 번째로 탐색해야할 자료가 유일한 형태를 의미
  - 선형구조는 주로 배열과 리스트의 형태로 저장
  - 일반적으로 1차원 배열에 자료를 저장하는 1차원 선형구조와 2차원 이상의 배열에 자료가 저장이 되어있는 다차원 선형구조로 구분(2차원, 3차원 구조라도 순서가 일정하게 정해져 있으면 이는 선형으로 볼 수 있음에 유의)

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 선형 구조의 탐색

- **순차 탐색** Sequential Search **시간복잡도: $O(n)$**
  - 전체탐색기법의 한 방법으로, 첫 번째 원소로부터 시작하여 한 원소씩 차례로 다음 원소를 탐색해 나가는 방법
  - 자료의 특성에 관계없이 사용할 수 있는 일반적인 방법

```
int sequential_search(int *data, int n, int key) {
                      탐색할 배열   탐색할 배열의 원소 수   찾고자 하는 값
  for(int i=0; i<n; i++)
    if(data[i]==key)     // 현재 원소가 찾는 값과 같다면
      return i;          // 현재 위치 반환
  return -1;             // 끝까지 찾지 못한 경우 -1 반환
}
```

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 선형 구조의 탐색

- **이분 탐색** Binary Search **시간복잡도: $O(\log n)$**
  - ✨정렬된 자료에서 가운데 값을 기준으로,탐색 범위를 절반씩 줄여 가며 원하는 값을 찾는 탐색 방법

0	1	2	3	4	5	6	7	8	9
5	16	21	25	34	56	61	65	72	99

```
int binary_search(int *data, int start, int end, int key){
  while(start<=end){
    int mid=(start+end)/2;
    if(data[mid]==key) return mid;
    if(data[mid]>key) end=mid-1;
    else start=mid+1;
  }
  return -1;
}
```

탐색할 배열 *탐색 범위의 시작 위치* *탐색 범위의 끝 위치* *찾고자 하는 값*

```
// 탐색 범위의 중앙 찾기
// 탐색 범위 중앙의 값과 비교
// 현재 원소가 찾는 값보다 크면
// 현재 원소가 찾는 값보다 작으면
// 끝까지 찾지 못한 경우 -1 반환
```

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 선형 구조의 탐색

- **이분 탐색 대표 문제**

0	1	2	3	4	5	6	7	8	9
5	16	21	25	34	70	70	70	83	99

```
int binary_search(int *data, int start, int end, int key){
  while(start<=end){
    int mid=(start+end)/2;
    if(          ?          ){
        result=mid;
        end=mid-1;
    }
    else
        start=mid+1;
  }
  return result;
}
```

lower_bound

key 이상이 처음 나오는 위치

`data[mid]>=key`

upper_bound

key 초과가 처음 나오는 위치

`data[mid]>key`

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 선형 구조의 탐색

- **순차 탐색과 이분 탐색 비교**

구분	순차 탐색	이분 탐색
자료 정렬 여부	정렬되지 않아도 탐색 가능	정렬된 상태에서만 탐색 가능
탐색 방식	하나씩 확인하며 탐색	절반씩 줄여 나가며 탐색
자료 수가 적을 때	적절	굳이 필요 없음
자료 수가 많을 때	느릴 수 있음	적절
시간복잡도	$O(n)$	$O(\log n)$

Copyright ⓒ 2026 by Lee San. All rights reserved.



STL

std:: → 이 기능은 내가 만든 것이 아니라
C++ 표준 라이브러리에서 가져온 것

- **STL 활용한 오름차순 정렬**

```
#include <stdio.h>
#include <algorithm>
using namespace std;   // 이미 존재하는 이름 공간을 사용! 앞으로 std::
생략!


int main(){
    int data[7] = {56,5,99,21,72,16,61};
    int cnt = 7;
std::
    sort(data, data+cnt);
    for(int i=0;i<cnt;i++) printf("%d ", data[i]);
    return 0;
}
```

Copyright ⓒ 2026 by Lee San. All rights reserved.



STL

- **STL 활용한 내림차순 정렬**

std:: → 이 기능은 내가 만든 것이 아니라 C++ 표준 라이브러리에서 가져온 것

```
#include <stdio.h>
#include <algorithm>
using namespace std;   // 이미 존재하는 이름 공간을 사용! 앞으로 std::
생략!
bool cmp(int x, int y){ return x > y; }   // x가 y보다 앞에 와야함.

int main(){
    int data[7] = {56,5,99,21,72,16,61};
    int cnt = 7;
std::
    sort(data, data+cnt, cmp);
    for(int i=0;i<cnt;i++) printf("%d ", data[i]);
    return 0;
}
```

Copyright ⓒ 2026 by Lee San. All rights reserved.

SASA OJ에서 다음 문제를 풀어볼까요?

[2026_2학기_프로그래밍I]

🔎 탐색기반설계 연습문제 #1



Copyright ⓒ 2026 by Lee San. All rights reserved.